Zero to Hero Study Handbook: agentic-aml-investigator

Module 1: Foundations & Architecture

1.1 What this project does

`agentic-aml-investigator` is a local AML investigation copilot built around:

- A DuckDB warehouse for account, transaction, alert, sanctions, evidence, and case-log data.
- A LangGraph investigation workflow (`src/aml_investigator/graph/build.py`) that moves one case from alert intake to final disposition.
- Deterministic forensic tools (`src/aml_investigator/tools/forensics.py`) that generate evidence rows and compact summaries.
- Local Ollama models through LangChain (`src/aml_investigator/llm.py`) for triage, risk assessment, and report drafting.
- Deterministic report validation and fallback generation (`src/aml_investigator/reporting.py`) to guarantee a valid final report.

Primary use cases in this repo:

- Investigate suspicious alerts end-to-end with human approval checkpoints.
- Evaluate model backbones and system reliability across labeled synthetic AML cases.
- Teach the system architecture via generated notebooks (`scripts/build_notebooks.py`).

1.2 Core paradigms and patterns used here

Definitions first:

- **State graph orchestration**: A directed workflow where each node updates a shared state dict. Used via `StateGraph` in `build_graph(...)`.
- **ReAct agent loop**: An LLM that reasons and calls tools iteratively. Used in `investigate(...)` with `create_agent(...)`.
- **Deterministic guardrail**: Non-LLM logic that enforces correctness/safety regardless of model behavior.
- **Structured output validation**: LLM output is validated into Pydantic schemas (`TriageDecision`, `RiskAssessment`, `JudgeScore`, `HumanDecision`).
- **Evidence-backed reporting**: Every report claim must reference stored evidence IDs (`EV-xx`) and grounded monetary values.
- **Hybrid pipeline pattern**: LLM for interpretation + deterministic SQL/rules/templates for reliability.

Patterns actually present in code:

- Functional, module-level orchestration functions (`triage`, `investigate`, `coverage_net`, `assess_risk`, etc.).

- Typed domain schemas with Pydantic and `Literal` constraints (`src/aml_investigator/schemas.py`).
- Deterministic data pipeline for synthetic ledger + OFAC loading (`src/aml_investigator/data/generator.py`, `src/aml_investigator/data/ofac.py`).
- Interrupt/resume human-in-the-loop workflow via LangGraph `interrupt(...)` + `Command(resume=...)`.
- Telemetry collection around LLM and tool calls (`src/aml_investigator/telemetry.py`).

1.3 Architecture and component interaction

Main components:

- **Configuration layer**: `Settings` in `src/aml_investigator/settings.py` (all runtime knobs with `AML_` env prefix).
- **Data layer**: `src/aml_investigator/db.py` + data generation/OFAC loaders.
- **Tool layer**: deterministic forensic tools and SQL guard.
- **LLM reliability layer**: `structured_llm_call(...)` escalation ladder.
- **Graph runtime layer**: `build_graph(...)` node topology and routing.
- **Evaluation layer**: case selection, batch runs, metrics, LLM judge.
- **Notebook authoring layer**: script-generated tutorial notebooks.

Main runtime flow (single case):

```
Input case {case_id, account_id, alerts}
    |
    v
triage (structured_llm_call -> TriageDecision)
    |
    v
investigate (ReAct agent + make_tools)
    |
    v
coverage_net (deterministic: run missing requested/mandatory checks)
    |
    v
assess_risk (structured_llm_call -> RiskAssessment)
    |                      \
    | needs_more_evidence \-> gather_more (bounded once) -> assess_risk
    v
write_report (LLM markdown)
    |
    v
validate_report (deterministic validation)
    | valid                    | invalid but retries left         | invalid and retries exha
    |                          |                                  |
    v                          v                                  v
```

```
human_gate (interrupt)      write_report (retry)              fallback_report (deterministic
    |
    v
finalize (persist report file + case_log row)
```

Secondary flow (evaluation):

```
eval_cases() -> run_eval() -> JSONL/CSV results -> decision_metrics/process_metrics
                                                \
                                                 -> judge_run() cross-family report rubric scores
```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`pyproject.toml`	Package metadata and dependencies	N/A	`requires-python`, `dependencies`, `dev` group
`README.md`	Project overview and quickstart	N/A	Quickstart commands, notebook order
`CLAUDE.md`	Project-specific operating conventions	N/A	Required commands and guardrails
`scripts/build_notebooks.py`	Programmatic generation of tutorial notebooks	`nb01`, `nb02`, `nb03`, `nb04`, `main`	`BUILDERS`, `NB_DIR`
`src/aml_investigation/config.py`	Configuration settings	`Settings`	`model_config` with `env_prefix="AML_"`, path/model/threshold settings
`src/aml_investigation/schemas.py`	Domain structured output schemas	`TriageDecision`, `RiskAssessment`, `JudgeScore`, `HumanDecision`, `RiskFactor`	`ALL_CHECKS`, `MANDATORY_CHECKS`, `CheckName`, `TypologyName`
`src/aml_investigation/tools.py`	DuckDB schema, connection, evidence store APIs	`connect`, `store_evidence`, `fetch_evidence`	`SCHEMA_DDL`, `AGENT_VISIBLE_TABLES`
`src/aml_investigation/ofac.py`	OFAC SDN download/load utilities	`download_sdn`, `load_sdn_frame`, `load_sdn_into`	`SDN_URL`, `SDN_COLUMNS`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`src/aml_investigator/data/generator.py`	Synthetic ledger generation + warehouse build	`Ledger`, `generate_ledger`, `build_warehouse`	`HIGH_RISK_COUNTRIES`, `ALERT_RULES_SQL`
`src/aml_investigator/tools/sql_guard.py`	SQL safety and anti-leakage enforcement for `run_sql`	`guard_sql`, `_literal_limit`, `SQLGuardError`	`_BANNED_SUBSTRINGS`, `AGENT_VISIBLE_TABLES`, `settings.sql_row_limit`
`src/aml_investigator/tools/forensics.py`	Deterministic forensic tool implementations	`make_tools`, `set_active_case`, `get_active_case`	`_ACTIVE_CASE`, `WAREHOUSE_SCHEMA_DOC`
`src/aml_investigator/llm.py`	Structured output reliability ladder and model factory	`get_chat_model`, `structured_llm_call`, `StructuredCallOutcome`	method order: `function_calling -> json_schema -> fallback`
`src/aml_investigator/prompts.py`	Agent system prompts	`TRIAGE_SYSTEM`, `INVESTIGATOR_SYSTEM`, `RISK_SYSTEM`, `REPORT_SYSTEM`	Prompt rules and typology doctrine
`src/aml_investigator/reporting.py`	Report validation and deterministic fallback report	`validate_report`, `render_fallback`, `_numbers_in`	`REQUIRED_SECTIONS`, `_MONEY_RE`, `_EVID_RE`
`src/aml_investigator/telemetry.py`	Case-level timing, token usage collection	`Telemetry`, `TelemetryEvent`, `case_scope`, `timed`, `LLMTimingHandler`	module-level `_state`, `_default`
`src/aml_investigator/graph/state.py`	Graph state contract	`InvestigationState`	Required/optional state keys
`src/aml_investigator/graph/build.py`	Full LangGraph construction and node logic	`build_graph`, node functions, routing helpers	`settings.max_reflection_rounds`, `settings.max_report_retries`
`src/aml_investigator/evaluation/cases.py`	Deterministic labeled eval case construction	`eval_cases`	`N_CLEAN`
`src/aml_investigator/evaluation/runner.py`	Batch execution of cases through graph	`run_eval`	JSONL resume behavior, `MemorySaver`
`src/aml_investigator/evaluation/metrics.py`	Decision/process/grounding metrics	`decision_metrics`, `process_metrics`, `report_groundedness`	Derived confusion/reliability metrics
`src/aml_investigator/evaluation/judge.py`	Cross-family LLM report judging	`judge_run`, `JUDGE_SYSTEM`	Judge output files `<run>_judge.jsonl/csv`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`tests/`	Behavioral contracts for core reliability logic	test functions by module	Guard, validator, metrics, structured-call expectations
`data/raw/sdn.csv`	Cached real OFAC SDN source data	N/A	Used by `load_sdn_frame`
`artifacts/`	Output directory for eval/reports/checkpoints	N/A	`artifacts/eval`, `artifacts/reports`, `artifacts/checkpoints`

Module 3: Core Execution Flows

Flow A: Data bootstrap (warehouse + alerts)

Entry points:

- `build_warehouse(force: bool = False)` in `src/aml_investigator/data/generator.py`
- Called explicitly from notebook code in `scripts/build_notebooks.py` (`nb01` cells).

Step-by-step:

1. `build_warehouse(...)` checks if `settings.warehouse_path` already exists.
2. Calls `download_sdn()` then `load_sdn_into(con)` to populate `sdn` table.
3. Calls `generate_ledger(seed=...)` to create `Ledger(accounts, transactions, ground_truth)`.
4. Inserts generated DataFrames into `accounts`, `transactions`, `ground_truth`.
5. Executes each SQL rule in `ALERT_RULES_SQL` and inserts alert rows into `alerts`.
6. Returns counts:

```
{"accounts": int, "transactions": int, "sdn": int, "alerts": int, "ground_truth": int}
```

Key input/output shapes:

- `generate_ledger(...) -> Ledger`
  - `Ledger.accounts`: `list[dict]` with keys `account_id, holder_name, country, account_type, opened_date`.
  - `Ledger.transactions`: keys `txn_id, ts, account_id, direction, txn_type, amount, counterparty_id, counterparty_name, counterparty_country, channel`.
  - `Ledger.ground_truth`: keys `account_id, typology, details`.

Flow B: Single-case investigation graph

Entry points:

- Graph build: `build_graph(con, checkpointer=None, model=None)` in `src/aml_investigator/graph/build.py`.
- Invocation examples are in notebook builder code (`nb03`) and eval runner (`run_eval`).

Initial state shape (`InvestigationState`):

```
{
  "case_id": str,
  "account_id": str,
  "alerts": [{"rule": str, "details": str}, ...]
}
```

Core node sequence and outputs:

1. `triage(state)`
   - Calls `structured_llm_call(TriageDecision, TRIAGE_SYSTEM, user, ...)`.
   - Writes state keys:

```
{
  "triage": {"priority": "high|medium|low", "checks": [...], "rationale": str},
  "reflection_rounds": 0,
  "report_retries": 0
}
```

2. `investigate(state)`
   - Creates ReAct-style agent from `create_agent(get_chat_model(...), list(tools.values()), ...)`.
   - Invokes with task text containing account, alerts, requested checks.
   - Reads evidence rows from `fetch_evidence(con, case_id)` and records completed checks.
3. `coverage_net(state)`
   - Deterministic safety net.
   - Requested set is `MANDATORY_CHECKS + triage checks`, or `ALL_CHECKS` for `manual_referral`.
   - Executes missing tool calls directly through `tools[name].invoke({"account_id": ...})`.
4. `assess_risk(state)`
   - Calls `structured_llm_call(RiskAssessment, RISK_SYSTEM, user, ...)`.
   - Applies deterministic sanctions guardrail:
     - If max sanctions hit score >= 93 and risk score < 75, force score floor to 75 and recommendation `ESCALATE`.
5. Optional reflection loop

    - `route_after_risk` routes to `gather_more` if:
        - `needs_more_evidence == True`
        - `requested_check` exists
        - `reflection_rounds < settings.max_reflection_rounds`
6. `write_report(state)`
    - Free-text LLM call with `REPORT_SYSTEM`.
    - Writes `report_md`.
7. `validate(state)`
    - Uses `validate_report(report_md, evidence, risk)`.
    - Updates `report_errors` and increments `report_retries` when invalid.
8. `fallback_report(state)` if retries exhausted
    - Uses deterministic `render_fallback(...)` template.
9. `human_gate(state)`
    - `interrupt(...)` payload includes `case_id`, `account_id`, `risk`, `report_md`, and a question.
    - Resume payload is normalized by `_normalize_human_decision(...)` into `HumanDecision`.
10. `finalize(state)`

- Computes final disposition (approved recommendation or override).
- Writes report file to `artifacts/reports/<safe_case_id>.md`.
- Inserts a row into `case_log`.

Important conditional routing:

- After risk: `assess_risk -> gather_more OR write_report`
- After validation: `validate -> human_gate OR write_report OR fallback_report`

Flow C: Forensic tool execution and evidence persistence

Entry point:

- `make_tools(con)` in `src/aml_investigator/tools/forensics.py`.

Tool set created:

- `profile_account`
- `velocity_scan`
- `structuring_scan`
- `counterparty_network`
- `sanctions_check`
- `run_sql`

Common behavior:

- Each tool is wrapped by `timed_tool(...)` for telemetry.
- On success, `_record(...)` stores full payload using `store_evidence(...)` and returns compact text:

`[EV-XX] <summary>`

`run_sql` specifics:

- Uses `guard_sql(query)` from `src/aml_investigator/tools/sql_guard.py`.
- Enforces single-statement read-only select semantics and table allowlist.
- If no `LIMIT`, wraps query with configured cap (`settings.sql_row_limit`, default 50).

Evidence row shape from `fetch_evidence(...)`:

```
{
  "evidence_id": "EV-01",
  "tool": str,
  "args": dict,
  "summary": str,
  "payload": dict
}
```

Flow D: Report validation and fallback guarantee

Entry points:

- `validate_report(report_md, evidence, risk)`
- `render_fallback(case_id, account_id, alerts, evidence, risk, retries)`

Validation checks (deterministic):

- Required headings: `## Case Summary`, `## Evidence`, `## Risk Assessment`, `## Recommendation`.
- Citation integrity: all `[EV-xx]` citations must exist in evidence IDs.
- Numeric groundedness: every dollar amount in markdown must map to a dollar value found in evidence summary/payload (with tolerance).
- Disposition line must match risk recommendation.

Return shape:

```
{"valid": bool, "errors": [str, ...]}
```

Fallback behavior:

- `render_fallback(...)` uses a Jinja template that always produces the required structure and disposition line.

Flow E: Evaluation pipeline

Entry points:

- `eval_cases(con)` -> labeled case list
- `run_eval(model, cases, run_name)` -> DataFrame + JSONL/CSV artifacts

- `decision_metrics(df)`, `process_metrics(df)`
- `judge_run(run_name, judge_model)`

Case shape from `eval_cases(...)`:

```
{
  "account_id": str,
  "label": "suspicious" | "clean",
  "typology": str,
  "alerts": [{"rule": str, "details": str}, ...]
}
```

`run_eval(...)` behavior:

- Compiles graph with `MemorySaver`.
- Executes each case with one retry budget for transient failures.
- Auto-resumes human gate with `{"decision": "approve"}`.
- Streams per-case rows to `artifacts/eval/<run_name>.jsonl`.
- Exports final CSV to `artifacts/eval/<run_name>_results.csv`.

Selected result fields written per case include:

- `status`, `error`, `account_id`, `label`, `typology_true`, `disposition`, `risk_score`, `typology_pred`
- `sanctions_hits_found`, `coverage_net_checks`, `coverage_net_failures`
- `used_fallback_report`, `report_retries`, `money_claims`, `grounded_claims`
- `wall_seconds`, `llm_calls`, `prompt_tokens`, `completion_tokens`, `tool_calls`, `tool_errors`, `structured_retries`, `structured_fallbacks`

Module 4: Setup & Run Guide

4.1 Prerequisites

From repository docs and manifests:

- Python `>=3.12,<3.13`
- `uv`
- Ollama running locally (default `http://localhost:11434`)
- Enough VRAM for local models (README notes roughly 8 GB for defaults)

4.2 Install and environment setup

Use `uv` only:

```
uv sync
```

Optional model pulls from `README.md`:

```
ollama pull granite4.1:8b
ollama pull qwen3.5:9b
```

Settings are loaded by Pydantic from env vars with `AML_` prefix and optional `.env` file (`SettingsConfigDict(env_prefix="AML_", env_file=".env")`).

4.3 Environment variables (.env keys)

The `Settings` class in `src/aml_investigator/settings.py` defines these overridable keys:

Env Var	Default	Purpose
`AML_AGENT_MODEL`	`granite4.1:8b`	Primary investigation model
`AML_JUDGE_MODEL`	`qwen3.5:9b`	Judge model
`AML_OLLAMA_BASE_URL`	`http://localhost:11434`	Ollama endpoint
`AML_TEMPERATURE`	`0.0`	LLM temperature
`AML_NUM_CTX`	`8192`	Context window budget
`AML_REQUEST_TIMEOUT`	`300.0`	Per-request timeout seconds
`AML_DATA_DIR`	`<repo>/data`	Data directory
`AML_WAREHOUSE_PATH`	`<repo>/data/aml.duckdb`	DuckDB file path
`AML_SDN_CSV_PATH`	`<repo>/data/raw/sdn.csv`	Cached OFAC CSV path
`AML_ARTIFACTS_DIR`	`<repo>/artifacts`	Output artifacts path
`AML_CHECKPOINT_DB_PATH`	`<repo>/artifacts/checkpoints/checkpoints.db`	LangGraph checkpoint SQLite path
`AML_SEED`	`42`	Ledger generation seed
`AML_N_ACCOUNTS`	`200`	Number of synthetic accounts
`AML_N_DAYS`	`90`	Ledger day span
`AML_LEDGER_END_DATE`	`2026-05-31`	Ledger end date
`AML_CTR_THRESHOLD`	`10000.0`	CTR threshold
`AML_STRUCTURING_BAND`	`0.85`	Structuring lower band multiplier
`AML_SANCTIONS_FUZZY_CUTOFF`	`80`	RapidFuzz cutoff
`AML_SQL_ROW_LIMIT`	`50`	Max rows for ad-hoc SQL
`AML_INVESTIGATOR_RECURSION_LIMIT`	`16`	ReAct recursion limit
`AML_MAX_REFLECTION_ROUNDS`	`1`	Risk reflection loop cap
`AML_MAX_REPORT_RETRIES`	`1`	Report retry cap

Minimal `.env` example:

```
AML_AGENT_MODEL=granite4.1:8b
AML_JUDGE_MODEL=qwen3.5:9b
AML_OLLAMA_BASE_URL=http://localhost:11434
```

4.4 Typical command sequences

Notebook-first workflow (as documented):

```
uv sync
uv run jupyter lab
```

Regenerate notebook files from source builder:

```
uv run python scripts/build_notebooks.py all
```

Execute notebooks (example from `CLAUDE.md`):

```
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/01_*.ipynb
```

Switch backbone for a run:

```
AML_AGENT_MODEL=qwen3.5:9b uv run jupyter lab
```

4.5 Database and seeding/migration notes

- There is no separate migration framework in this repo.
- `db.connect(read_only=False)` executes `SCHEMA_DDL` and creates tables/indexes if missing.
- `build_warehouse(force=True)` is the canonical seed/reset path:
  - Downloads/caches OFAC SDN CSV.
  - Regenerates synthetic ledger and ground-truth labels.
  - Rebuilds alerts table.
- Regenerating warehouse can reshuffle account IDs, so notebook code fetches account IDs from `ground_truth` dynamically.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. Start with `README.md` and `CLAUDE.md` to understand the project goal, constraints, and command flow.
2. Read `src/aml_investigator/settings.py`, `src/aml_investigator/schemas.py`, and `src/aml_investigator/graph/state.py` to learn core contracts and data types.
3. Read `src/aml_investigator/db.py` and `src/aml_investigator/tools/sql_guard.py` to understand data boundaries and leakage prevention.
4. Read `src/aml_investigator/data/generator.py` and `src/aml_investigator/data/ofac.py` to understand dataset creation and alert derivation.
5. Read `src/aml_investigator/tools/forensics.py` to understand deterministic evidence generation and tool contracts.
6. Read `src/aml_investigator/llm.py` and `src/aml_investigator/prompts.py` to understand structured output and prompt doctrine.
7. Read `src/aml_investigator/reporting.py` to understand report groundedness guarantees.

8. Deep-dive `src/aml_investigator/graph/build.py` end-to-end.
9. Finish with `src/aml_investigator/evaluation/*.py` and `scripts/build_notebooks.py` for full lifecycle understanding.

5.2 Practice exercises (with solution outlines)

Exercise 1 Question: In a `manual_referral` case, why can the graph still run all five checks even if triage chooses fewer checks?

Model answer outline:

- In `coverage_net(...)`, requested checks start as `MANDATORY_CHECKS + triage checks`.
- If any alert has `rule == "manual_referral"`, requested set is replaced with `ALL_CHECKS`.
- Missing checks are executed deterministically via `tools[name].invoke(...)`.

Exercise 2 Question: Trace where a report file path is created and how path traversal is prevented.

Model answer outline:

- In `finalize(...)` in `graph/build.py`, `report_dir = settings.artifacts_dir / "reports"`.
- Filename is sanitized using `_safe_case_filename(case_id)`.
- `report_path = (report_dir / filename).resolve()` is checked so `report_dir.resolve()` must be in `report_path.parents`.
- Otherwise it raises `ValueError`.

Exercise 3 Question: What exact conditions cause `validate_report(...)` to fail?

Model answer outline:

- Missing any required heading from `REQUIRED_SECTIONS`.
- Cited evidence ID not in known IDs.
- No evidence citations at all.
- Any dollar figure not grounded in evidence/risk score numeric pool.
- Missing or mismatched `DISPOSITION: <recommendation>` line.

Exercise 4 Question: How does `guard_sql(...)` enforce safe ad-hoc SQL and row limits?

Model answer outline:

- Rejects banned substrings (`attach`, `install`, `copy`, etc.).
- Parses SQL with `sqlglot` and requires exactly one statement.
- Allows only `exp.Select` or `exp.Union`.
- Rejects tables outside `AGENT_VISIBLE_TABLES`.

- If no limit, wraps query with `LIMIT settings.sql_row_limit`.
- Caps oversized limits and rejects non-literal limits.

Exercise 5 Question: What does `structured_llm_call(...)` do when the model returns invalid structured output twice?

Model answer outline:

- Attempt 1: `function_calling`.
- Attempt 2: `json_schema` with prior error feedback appended.
- If both fail and `fallback` exists, returns `StructuredCallOutcome(..., method_used="fallback", attempts=3)`.
- If no fallback, raises `RuntimeError`.

Exercise 6 Question: Explain how sanctions evidence can force escalation even when model risk score is low.

Model answer outline:

- In `assess_risk(...)`, sanctions hit scores are extracted from evidence payload where `tool == "sanctions_check"`.
- If `max_hit >= 93` and predicted `risk_score < 75`, code updates risk with score floor 75 and recommendation `ESCALATE`.
- It appends a guardrail `RiskFactor` referencing sanctions evidence ID.

Exercise 7 Question: What is the shape of one eval-case input and one eval result row?

Model answer outline:

- Eval case from `eval_cases(...)`:
  - `account_id`, `label`, `typology`, `alerts: [{rule, details}]`.
- Result row from `run_eval(...)` includes:
  - outcome fields (`disposition`, `risk_score`, `typology_pred`),
  - reliability fields (`tool_errors`, `structured_retries`, `used_fallback_report`),
  - groundedness fields (`money_claims`, `grounded_claims`),
  - runtime/token fields (`wall_seconds`, `prompt_tokens`, `completion_tokens`).

Exercise 8 Question: Where would you add a new deterministic forensic check so both the ReAct investigator and coverage net can use it?

Model answer outline:

- Implement the tool function in `make_tools(...)` in `tools/forensics.py` and include it in returned mapping.
- Add the check name to `CheckName` / `ALL_CHECKS` and optionally `MANDATORY_CHECKS` in `schemas.py`.
- Update triage prompt check vocabulary (`TRIAGE_SYSTEM`) and investigator instructions if needed.

- Ensure any SQL references remain within allowed tables or update guard policy deliberately.

Learner Verification Checklist

Use this to self-check understanding:

- Can you explain the full graph path from `triage` to `finalize`, including both conditional routes?
- Can you describe the exact evidence lifecycle (`store_evidence` -> `fetch_evidence` -> citations in report)?
- Can you explain why `manual_referral` changes check coverage behavior?
- Can you explain `structured_llm_call` escalation and when deterministic fallback is used?
- Can you list what `validate_report` enforces and why this blocks hallucinated dollar claims?
- Can you explain how `guard_sql` prevents leakage of `ground_truth`, `evidence`, and `case_log`?
- Can you reconstruct the key columns emitted by `run_eval` and what each reliability metric means?
- Can you point to where human override is normalized/validated and persisted?
- Can you explain how and where checkpointing supports interrupt/resume in notebook flow?
- Can you identify which settings are safe to tune first (model, cutoff, row limit, retry caps) and where they live?